3-5 Cache

- 1. 局部性原理
  - 基本概念
    - **局部性原理**：程序访问存储器时，短时间内往往集中访问一小部分地址。
    - **Cache 依据**：把近期常用的数据或指令复制到高速缓冲中，让 CPU 尽量直接访问 Cache。
  - 时间局部性
    - **含义**：某个存储单元刚被访问后，在较短时间内很可能再次被访问。
  - 空间局部性
    - **含义**：某个存储单元被访问后，它附近的单元在较短时间内也可能被访问。
    - **注意**：空间局部性对单个变量意义不大。
- 2. Cache 的工作原理
  - Cache 与主存
    - **Cache 内容**：存放主存中被频繁访问的程序和数据副本。
    - **容量关系**：Cache 只保存主存的一部分内容，容量远小于主存。
    - **交换单位**：主存和 Cache 之间以块为单位交换信息。
    - **主存块**：主存被划分成若干块，每块包含多个存储单元。
    - **Cache 行**：Cache 中存放一个主存块的位置，也称 Cache 块。
  - 地址结构
    - **Cache 地址**：`Cache 行号 + 块内地址` 。
    - **主存地址**：`主存块号 + 块内地址` 。
    - **块内地址相同**：主存块和 Cache 行大小相同，因此块内地址位数相同。
    - **位数关系**：若 Cache 有 `2^c` 行，块内有 `2^b` 个存储单元，则 Cache 地址可分为 `c 位行号 + b 位块内地址` 。
    - **主存更大**：若主存有 `2^m` 块，则主存地址可分为 `m 位主存块号 + b 位块内地址` ，通常 `c < m` 。
  - 有效位
    - **作用**：说明某个 Cache 行中的信息是否有效。
    - **初始状态**：系统启动或复位时，Cache 行为空，有效位为 0。
    - **装入后**：主存块装入 Cache 行后，有效位变为 1。
    - **冲刷方式**：把有效位改为 0，可让该行信息失效；物理上不一定删除数据。
    - **位置**：有效位是 Cache 行额外设置的控制位，主存块中没有。
  - CPU 访问过程
    - **给出地址**：CPU 给出主存地址 AD。
    - **判断命中**：检查 AD 所在主存块是否已在 Cache 中。
    - **命中**：直接从 Cache 中取信息送 CPU。
    - **缺失**：从主存取出 AD 所在的整块内容。
    - **装入 Cache**：在 Cache 中找到空闲行或替换行，将主存块复制到 Cache。
    - **送 CPU**：同时把 AD 对应存储单元内容送 CPU。
  - 平均访问时间
    - **命中率 p**：命中次数 / 总访问次数。
    - **命中时间 Tc**：命中时访问 Cache 的时间，包含判断命中/缺失的时间。
    - **缺失代价 Tm**：缺失时从主存读取主存块并送入 Cache 的额外时间。
    - **未命中时间**：`Tm + Tc` 。
    - **平均访问时间**：`Ta = p x Tc + (1 - p) x (Tm + Tc) = Tc + (1 - p) x Tm` 。
- 3. Cache 行与主存块的映射方式
  - 映射方式总览
    - **直接映射**：每个主存块只能映射到 Cache 的固定行。
    - **全相联映射**：每个主存块可以映射到 Cache 的任意行。
    - **组相联映射**：每个主存块映射到 Cache 的固定组，组内可放任意行。
  - 直接映射
    - **映射规则**：`Cache 行号 = 主存块号 mod Cache 行数` 。
    - **地址划分**：主存地址划分为 `tag + Cache 行号 + 块内地址` 。
    - **tag 位数**：若主存块号为 `m` 位，Cache 行号为 `c` 位，则 `tag = m - c` 位。
    - **Cache 行内容**：包括有效位、tag、一致性维护位、替换算法位和数据。
    - **常见误区**：Cache 行号和块内地址只是地址划分，不会作为字段存入 Cache 行；tag 会存入 Cache 行，用来比较。
  - 直接映射访存过程
    - **定位行**：根据主存地址中的 `Cache 行号` 字段找到唯一对应行。
    - **比较 tag**：比较地址中的 tag 与该 Cache 行保存的 tag。
    - **检查有效位**：tag 相等且有效位为 1，则 Cache 命中。
    - **读取单元**：根据块内地址定位 Cache 行中的具体存储单元。
    - **缺失处理**：若 tag 不等或有效位不是 1，则从主存读出整块，送入对应 Cache 行，设置有效位为 1，并把 tag 置为地址高位 tag。
  - 全相联映射
    - **映射规则**：主存块可放入 Cache 任意行。
    - **地址划分**：主存地址划分为 `tag + 块内地址` 。
    - **tag 位数**：tag 就是完整主存块号，长度为 `m` 位。
    - **查找方式**：需要在多个 Cache 行中比较 tag，判断是否命中。
  - 组相联映射
    - **映射规则**：`Cache 组号 = 主存块号 mod Cache 组数` 。
    - **本质**：组间直接映射 + 组内全相联。
    - **x 路组相联**：一组内有 x 个 Cache 行。
    - **地址划分**：主存地址划分为 `tag + Cache 组号 + 块内地址` 。
    - **命中判断**：先由组号定位组，再在组内比较多个 tag。
  - 映射方式比较
    - **关联度**：一个主存块映射到 Cache 中时，可能存放的位置个数。
    - **直接映射关联度**：1。
    - **全相联关联度**：Cache 行数。
    - **组相联关联度**：组路数。
    - **命中率**：关联度越低，命中率越低；直接映射最低，全相联最高。
    - **命中时间**：关联度越低，判断开销越小；直接映射最短，全相联最长。
    - **比较器个数**：直接映射 1 个，全相联为 Cache 行数个，组相联为组路数个。
    - **比较器位数**：等于 tag 位数。
- 4. Cache 中主存块的替换方式
  - 替换原因
    - **容量差异**：Cache 行数远少于主存块数。
    - **冲突情况**：多个主存块可能映射到同一行或同一组。
    - **替换条件**：新主存块要装入时，目标位置已满，就必须淘汰一个旧块。
  - 常见替换算法
    - **FIFO**：先进先出，总是替换最早装入 Cache 的主存块。
    - **LRU**：最近最少使用，总是替换近期最少使用的主存块。
    - **LFU**：最不经常使用，替换引用次数最少的块。
    - **随机替换**：从候选行中随机选择一个淘汰。
  - LRU 计数器
    - **计数含义**：计数器值越高，说明该行越久未被使用。
    - **命中时**：被访问行计数器清零，比它低的计数器加 1，其余不变。
    - **未命中且有空闲行**：新装入行计数器置 0，其余全加 1。
    - **未命中且无空闲行**：淘汰计数值最大的行，新装入行计数器置 0，其余加 1。
    - **LRU 位数**：`ceil(log2(路数))` 。
- 5. Cache 的一致性问题
  - 问题来源
    - **副本关系**：Cache 中的数据是主存块副本。
    - **写操作风险**：如果只更新一处，Cache 与主存可能不一致。
    - **分类**：写命中策略和写不命中策略。
  - 写命中策略
    - **全写法**：写命中时，同时写 Cache 和主存。
    - **回写法**：写命中时，只写 Cache，不立即写主存；该块被替换出时才写回主存。
    - **修改位/脏位**：回写法下，每个 Cache 行需要 1 位修改位；修改位为 1 表示该行被修改过，替换时必须写回主存。
  - 写不命中策略
    - **写分配法**：先在主存块中更新相应单元，再分配 Cache 行，把更新后的主存块装入 Cache。
    - **非写分配法**：只更新主存单元，不把主存块装入 Cache。
  - 常见搭配
    - **回写法 + 非写分配法**：第一次写不命中后不装入 Cache，后续对该块写操作仍可能一直不命中，持续写主存。
    - **回写法 + 写分配法**：第一次写不命中后装入 Cache，后续对该块写操作命中，持续写 Cache。
    - **效率倾向**：回写法通常搭配写分配法。
    - **策略关系**：课件中强调写命中策略和写不命中策略可以按题目要求搭配。

by joshuji 由 幕布 发布